Laboratório 3 — Dados relacionais e operações do tipo *join*

ME315

João Vitor Matoso

2026-08-31

```
library(readr)
library(dplyr)
library(stringr)
library(tidyr)

    cat("Documento compilado em:",
    format(Sys.time(), "%Y-%m-%d %H:%M:%S", tz = "America/Sao_Paulo"), "\n")
```

```
Documento compilado em: 2026-08-31 21:37:20
```

1. Importação dos dados

```
airlines <- read_csv("airlines.csv")

airports <- read_csv(
  "airports.csv",
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
)
```

Para `flights`, o arquivo é lido em blocos de 1 milhão de linhas. Em cada bloco: removemos vôos cujo aeroporto de destino começa com "1", removemos linhas com dados faltantes, e calculamos as estatísticas suficientes (soma dos atrasos e número de vôos) por aeroporto de destino. Ao final, essas estatísticas são combinadas entre todos os blocos para produzir o atraso médio por aeroporto.

```r
processa_bloco <- function(bloco, pos) {
  bloco %>%
    filter(!str_starts(DESTINATION_AIRPORT, "1")) %>%
    drop_na() %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atraso = sum(ARRIVAL_DELAY),
      n_voos = n(),
      .groups = "drop"
    )
}

resultados_por_bloco <- read_csv_chunked(
  file = "flights.csv",
  callback = DataFrameCallback$new(processa_bloco),
  chunk_size = 1e6,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

flights <- resultados_por_bloco %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma_atraso = sum(soma_atraso),
    n_voos = sum(n_voos),
    .groups = "drop"
  ) %>%
  mutate(atraso_medio = soma_atraso / n_voos)
```

Resultados intermediários por bloco:

```r
resultados_por_bloco
```

```
# A tibble: 1,874 x 3
   DESTINATION_AIRPORT soma_atraso n_voos
   <chr>                     <dbl>  <int>
 1 ABE                        3413    350
 2 ABI                        5926    446
 3 ABQ                       20448   3241
```

```
 4 ABR                          -645    124
 5 ABY                          1339    169
 6 ACT                          4436    271
 7 ACV                          2748    294
 8 ACY                          9069    701
 9 ADK                          -293     19
10 ADQ                            72     59
# i 1,864 more rows
```

Tabela final `flights`:

```
flights
```

```
# A tibble: 322 x 4
   DESTINATION_AIRPORT soma_atraso n_voos atraso_medio
   <chr>                     <dbl>  <int>        <dbl>
 1 ABE                       12874   2220         5.80
 2 ABI                        9077   2224         4.08
 3 ABQ                      106415  18953         5.61
 4 ABR                       -2227    657        -3.39
 5 ABY                        7501    864         8.68
 6 ACK                        2393    487         4.91
 7 ACT                       16392   1541        10.6
 8 ACV                       12259   1267         9.68
 9 ACY                       41744   3530        11.8
10 ADK                        -284     89        -3.19
# i 312 more rows
```

2. Combinando `flights` com `airports`

a. A coluna chave em `flights` é `DESTINATION_AIRPORT`, que contém o código IATA do aeroporto de destino do vôo.

b. A coluna chave em `airports` é `IATA_CODE`, que contém o código IATA de cada aeroporto.

c. Aeroportos que aparecem em `flights` mas estão ausentes em `airports`:

```
anti_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

```
# A tibble: 0 x 4
# i 4 variables: DESTINATION_AIRPORT <chr>, soma_atraso <dbl>, n_voos <int>,
#   atraso_medio <dbl>
```

d-e. Combinando as tabelas com as chaves explícitas e armazenando o resultado em `flights`:

```
flights <- left_join(
  flights,
  airports %>% select(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE),
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)

flights
```

```
# A tibble: 322 x 8
   DESTINATION_AIRPORT soma_atraso n_voos atraso_medio CITY       STATE LATITUDE
   <chr>                     <dbl>  <int>        <dbl> <chr>      <chr>    <dbl>
 1 ABE                       12874   2220         5.80 Allentown  PA        40.7
 2 ABI                        9077   2224         4.08 Abilene    TX        32.4
 3 ABQ                      106415  18953         5.61 Albuquerq~ NM        35.0
 4 ABR                       -2227    657        -3.39 Aberdeen   SD        45.4
 5 ABY                        7501    864         8.68 Albany     GA        31.5
 6 ACK                        2393    487         4.91 Nantucket  MA        41.3
 7 ACT                       16392   1541        10.6  Waco       TX        31.6
 8 ACV                       12259   1267         9.68 Arcata/Eu~ CA        41.0
 9 ACY                       41744   3530        11.8  Atlantic ~ NJ        39.5
10 ADK                        -284     89        -3.19 Adak       AK        51.9
# i 312 more rows
# i 1 more variable: LONGITUDE <dbl>
```

3. Número de aeroportos por estado

```
contagem_estado <- airports %>%
  group_by(STATE) %>%
  summarise(numero_aeroportos = n()) %>%
  arrange(desc(numero_aeroportos))

contagem_estado
```

```
# A tibble: 54 x 2
   STATE numero_aeroportos
   <chr>             <int>
 1 TX                   24
```

```
 2 CA                   22
 3 AK                   19
 4 FL                   17
 5 MI                   15
 6 NY                   14
 7 CO                   10
 8 MN                    8
 9 MT                    8
10 NC                    8
# i 44 more rows
```

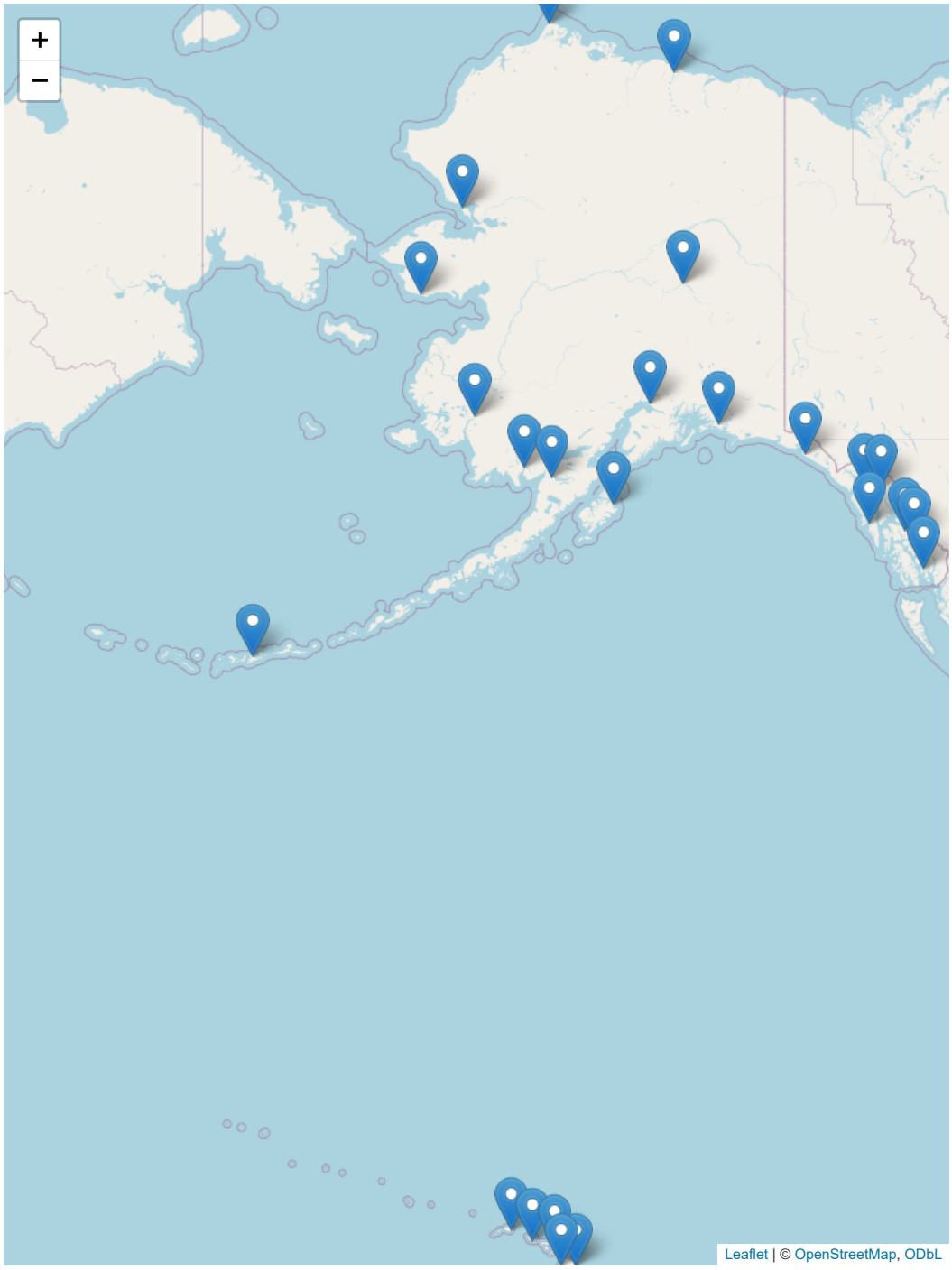
4. Mapa dos atrasos por aeroporto

```
library(leaflet)

flights_mapa <- flights %>%
  drop_na(LATITUDE, LONGITUDE)

this <- leaflet(flights_mapa) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste0(
      DESTINATION_AIRPORT, " - ",
      "atraso médio: ", round(atraso_medio, 1), " min (",
      n_voos, " vôos)"
    )
  )

this
```


+
−
Leaflet | © OpenStreetMap, ODbL